Relatório Técnico de Trabalho Prático

Compiladores: Implementação do Front-end para Subset de Ada

David Sá (up202303508)
João Morais (up202307077)

Outubro 2025

Conteúdo

1 Introdução **3**

2 Método e Instrumentos **3**
2.1 Fluxo de Compilação 3
2.2 Estruturas de Dados Chave 3
2.2.1 Árvore Sintática Abstrata (AST) 3
2.2.2 Tabela de Símbolos 3

3 Implementação do Subset de Ada **3**
3.1 Estrutura do Programa e Declarações 4
3.2 Comandos e Controlo de Fluxo 4
3.3 Expressões 4

4 Instruções de Submissão e Execução **4**

5 Conclusão **4**

1 Introdução

Este documento técnico descreve a realização da primeira etapa da atividade prática da disciplina de Compiladores (DCC-FCUP). O objetivo principal é desenvolver o front-end de um compilador para uma versão simplificada da linguagem de programação Ada, conforme descrito. O objetivo principal desta entrega é criar o Analisador Léxico (Scanner) e implementar um Analisador Sintático (Parser) com a capacidade de gerar a Árvore Sintática Abstrata (AST) a partir do código-fonte fornecido.

2 Método e Instrumentos

O projeto foi implementado em linguagem C, utilizando as ferramentas padrão do ecossistema GNU para a construção de compiladores, especificamente o `Flex` e o `Bison`.

2.1 Fluxo de Compilação

O sistema segue o fluxo típico de um compilador:

1. **Análise Léxica (Scanner):** O `Flex` gera o `yylex()`, que lê o código-fonte e o transforma numa sequência de *tokens*.
2. **Análise Sintática (Parser):** O `Bison` utiliza a gramática definida em `parser.bison` para verificar a correção sintática da sequência de *tokens*.
3. **Geração da AST:** À medida que o Parser reconhece as regras da gramática, as *ações semânticas* do `Bison` chamam as funções de construção da AST (`ast_...`), representadas em `ast.h` e `ast.c`.

2.2 Estruturas de Dados Chave

2.2.1 Árvore Sintática Abstrata (AST)

A AST é a estrutura de dados central para representar a lógica do programa. É composta por nós de `Expr` (Expressão) e `Command` (Comando). Esta representação elimina pormenores sintáticos (como parêntesis ou ponto e vírgulas) e foca-se apenas no significado.

2.2.2 Tabela de Símbolos

Implementada em `symboltable.h` e `symboltable.c`, esta estrutura é vital para a gestão de variáveis.

- É uma **tabela *hash* com *chaining*** para resolver colisões.
- Utiliza um mecanismo de **pilha de *scopes*** para gerir o aninhamento de blocos, embora nesta fase se utilize primariamente para o *scope* global.
- Permite a verificação imediata de **variáveis não declaradas** (`symtab_lookup()`) e previne **redeclarações** no mesmo *scope* (`symtab_insert()`).

3 Implementação do Subset de Ada

O Parser (`parser.bison`) foi concebido para suportar na íntegra o *subset* de Ada definido no enunciado.

3.1 Estrutura do Programa e Declarações

A regra principal `program` segue a sintaxe exigida:

```
PROCEDURE MAIN IS
  <decls>
BEG
  <cmd_list>
END MAIN;
```

Listing 1: Estrutura Base

O `Bison` insere (`symtab_insert`) as variáveis na Tabela de Símbolos durante a fase `decls`, garantindo que as declarações estejam corretas antes de processar a lista de comandos.

3.2 Comandos e Controlo de Fluxo

O Parser suporta os comandos de controlo de fluxo pedidos, gerando o nó `Command` apropriado na AST:

- **Atribuição:** `ID := expr ;` (gera o nó `E_ASSIGNMENT` encapsulado em `EXPR_CMD`).
- **Condicional:** `IF expr THEN cmd_list ELSE cmd_list END IF ;` (gera o nó `IF_ELSE_CMD`).
- **Loop:** `WHILE expr LOOP cmd_list END LOOP ;` (gera o nó `WHILE_CMD`).
- **I/O:** `PUT_LINE expr ;` e `GET_LINE ID ;` (gera `PUT_LINE_CMD` e `GET_LINE_CMD`).

3.3 Expressões

As expressões aritméticas e booleanas são processadas com a devida consideração da precedência de operadores:

- Foi utilizada a declaração `%left` no `parser.bison` para resolver os conflitos *shift/reduce* e definir a ordem de avaliação correta (`MULT/DIV` antes de `PLUS/MINUS`).
- As expressões são mapeadas para os nós `E_OPERATION` (aritméticas) ou `E_BOOL` (comparações) na AST.

4 Instruções de Submissão e Execução

O trabalho será submetido num ficheiro `COMP TP1 GRUPO n.zip`, contendo todo o código-fonte (`.c`, `.h`, `.l`, `.bison`) e este relatório (`README.pdf`).

Para compilar e executar o projeto:

```
# 1. Compilar o projeto (o Makefile gera o executavel "interpreter")
make

# 2. Executar o interpretador (que utiliza o parser/AST)
./interpreter <nome\_do\_ficheiro>.ada
```

A fase de execução neste projeto é tratada pelo ficheiro `interpreter.c`, que percorre a AST gerada pelo Parser.

5 Conclusão

A primeira fase do trabalho de Compiladores está funcionalmente concluída. O Front-end foi efetivamente implementado, assegurando a Análise Léxica, a Análise Sintática e a criação da AST para o *subset* da linguagem Ada definido. As etapas seguintes do projeto poderão concentrar-se na Análise Semântica (verificação de tipos) e na Geração de Código.